

WILLKOMMEN BEI MEINEM MRP PROJEKT - PROTOKOLL

1. BESCHREIBUNG DER TECHNISCHEN SCHRITTE UND ARCHITEKTURENTSCHEIDUNGEN

Ich habe das Projekt schichtbasiert in Java umgesetzt. Ziel war eine klare Trennung von Verantwortlichkeiten sowie eine gut wartbare und erweiterbare Struktur.

Meine Architektur gliedert sich in folgende Hauptkomponenten:

1.1 HANDLER-SCHICHT:

Verantwortlich für die Verarbeitung von HTTP-Requests.
Diese Schicht übernimmt:

- Validierung von URL, HTTP-Methode und Request-Body
- Authentifizierung des aktuellen Benutzers
- Mapping von JSON zu Java-Objekten
- Rückgabe geeigneter HTTP-Statuscodes

1.2 SERVICE-SCHICHT:

Enthält die fachliche Logik der Anwendung.
Hier werden:

- Berechtigungen geprüft
- Business Logic umgesetzt (z. B. unveränderbare Felder)
- Datenbankoperationen koordiniert (Repositories aufgerufen)

1.3 REPOSITORY-SCHICHT:

Verantwortlich für den direkten Datenbankzugriff.
SQL-Statements sind klar gekapselt, und Methoden geben nur einfache Rückgabewerte (true/false) zurück.

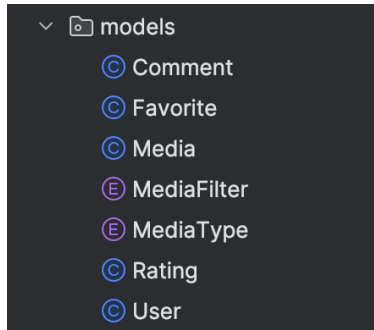
Zusätzlich wurde bewusst entschieden:

- Sicherheitsrelevante Informationen (z. B. creatorId) nicht aus dem Request-Body, sondern ausschließlich aus dem Auth-Kontext zu beziehen
- UNveränderbare Felder (z. B. title, creator_id) serverseitig zu schützen
- UUIDs als Primärschlüssel zu verwenden

1.4 MODEL KLASSEN:

Meine Model Klassen helfen dabei von überall aus auf die „Modelle“ zugreifen zu können. Ich definiere darin welche Parameter die einzelnen Typen haben. zB: id, title, media_type usw.

Und das jeweils für

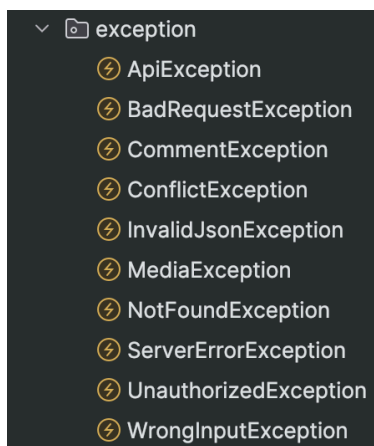


1.5 EXCEPTIONS

Ich habe eigene Exception Types definiert, um für jeden Fall eine eigene Response geben zu können, die das Problem möglichst gut beschreibt. Dies hilft, dass der User oder der Front End Developer sieht was das Problem ist.

Ich habe eine ApiException Klasse gemacht, von der dann die anderen Klassen erben. Es gibt immer einen HTTP-Status und einen Error Code.

Also zB: 409, COMMENT_EXISTS



1.6 DTO KLASSEN (DATA TRANSFER OBJECT)

Ich habe DTOs implementiert um die Daten aus mehreren „Repos“ zusammenfügen zu können. Wenn ich zum Beispiel ein Medium mit den dazugehörigen average Rating abrufen will mache ich das über mein MediaWithRatingDto. Hier werden die Daten zusammengehängt und dann ausgegeben. Ich kann somit mein Media Model so lassen wie es ist und wenn ich will die Ratings auch anzeigen. Ein anderes Bsp ist das Leaderboard. Hier werden auch viele Daten aus unterschiedlichen Quellen reduziert in einer eigenen Form dargestellt. Das DTO hilft dabei. Es werden nicht immer alle, sondern nur die relevanten Daten übertragen. (Leaderboard braucht nur username und nicht alle anderen Daten der User, dafür aber Daten aus anderen Quellen)

2. UNIT TESTS:

Bei den Unit Tests habe ich darauf geachtet, die Service-Klassen isoliert zu testen. Alle externen Abhängigkeiten (Repositories und weitere Services) wurden mit Mockito gemockt, um Datenbankzugriffe zu vermeiden.

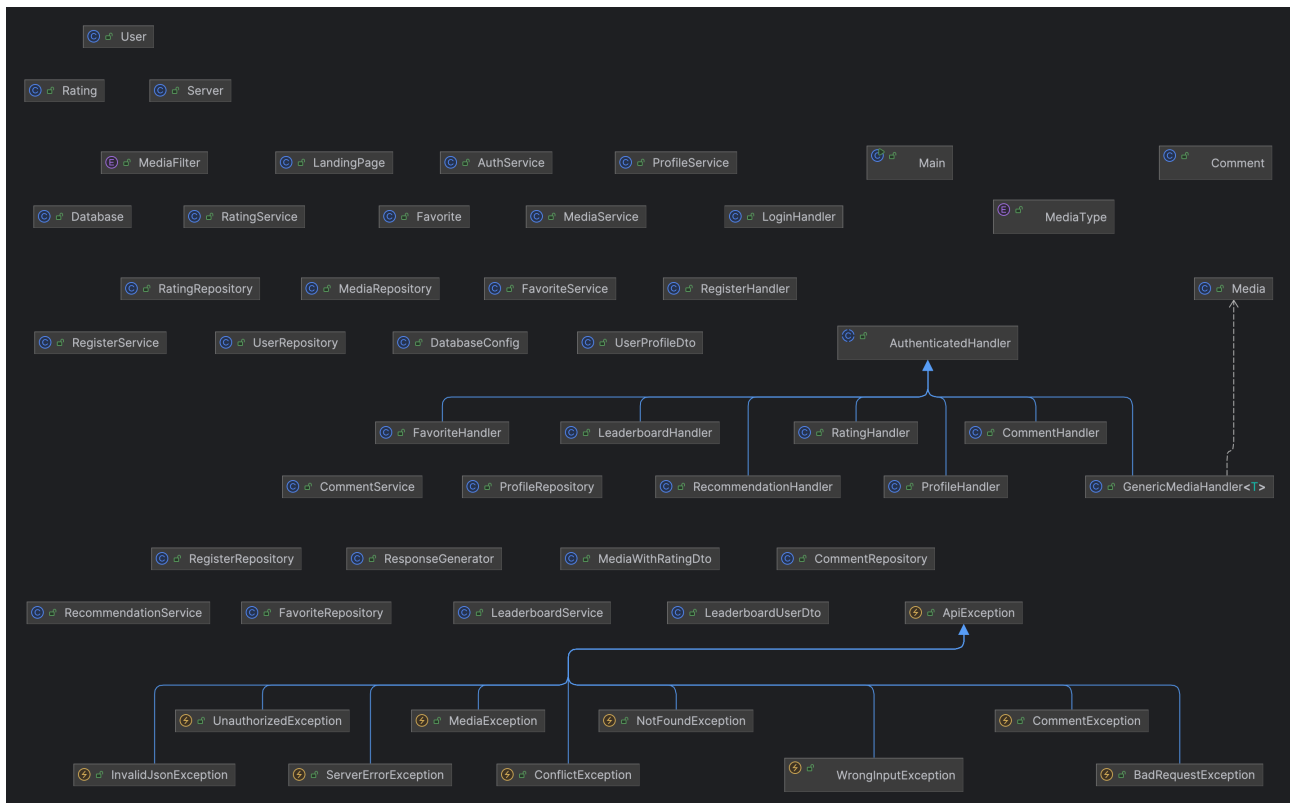
Getestet wurden insbesondere:

- Validierungslogik (z. B. Benutzername- und Passwortregeln)
- Erfolgs- und Fehlerfälle (z. B. Benutzer existiert bereits / Benutzer nicht gefunden)
- Korrekte Delegation von Methodenaufrufen an abhängige Services und Repositories
- Rückgabewerte bei normalen, leeren und fehlerhaften Ergebnissen
- Sicherheitsrelevante Aspekte, wie das Hashen von Passwörtern vor dem Speichern
- Die Funktion meiner Data Transfer Objects (Getter und Setter)

Zusätzlich wurde überprüft, dass:

- nur die erwarteten Abhängigkeiten aufgerufen werden
- keine unnötigen Interaktionen stattfinden
- private Felder und DTOs ausschließlich über Getter verwendet werden

3. UML



4. AUFGETRETENE PROBLEME UND DEREN LÖSUNG

Während der Entwicklung hatte ich mehrere klassische Probleme...

- Ich beschäftige mich in meiner Freizeit hauptsächlich mit Frontend und Client Side (React und React Native). Demnach habe ich noch nie ein Backend/Server in diesem Ausmaß geschrieben und musste noch nicht so gezielt auf Layered Architekture achten, aber dieses Projekt war unfassbar hilfreich was das angeht. Ich verstehe jetzt viel besser wieso ein guter und klarer Aufbau des Codes so wichtig ist.
- Zu Beginn habe ich Front und Backend vermischt und HTML in meinen Backend Code eingebaut, was natürlich ein totaler Blödsinn war.
- Ein klassischer Fehler, den ich leider viel zu oft bei Projekten mache war, dass ich direkt drauf los gearbeitet habe ohne mir davor viele Gedanken über die Umsetzung zu machen. Wie IMMER hat das dazu geführt, dass alles sehr schnell sehr unübersichtlich wurde. Außerdem hatte ich zuvor noch nie mit Java zutun und habe mir ein paar Sachen von ChatGPT erklären und schreiben lassen, was dann zu großen Problemen geführt hat. Irgendwann habe ich mich in meinem eigenen Code nicht mehr ausgekannt und habe mich dazu entschlossen von neu anzufangen, weil ich keinen Sinn mehr darin gesehen habe weiterzumachen und Java wirklich lernen und verstehen wollte. Nach sehr vielen Stunden und einigen Nervenzusammenbrüchen kann ich jetzt sagen, dass ich sehr froh bin neu angefangen zu haben.
- Zu den technischen Problemen kann ich sagen, dass ich mit Docker etwas zu kämpfen hatte. Das Setup hat bei mir leider etwas länger gedauert als ich mir gewünscht hätte. Nach diesen Startschwierigkeiten läuft jetzt aber alles problemlos.
- Außerdem wäre es meiner Ansicht nach schlau gewesen, wenn ich vor Beginn der Programmierung schon ein Datenbankmodell erstellt hätte. Im Nachhinein musste ich dann relativ viel im Code umschreiben, damit meine Klassen und Methoden zu meiner DB passen.
- Ein ähnliches Problem hatte ich mit Exceptions und JSON Responses. Ich habe relativ früh einen ResponseHandler erstellt, aber gegen Ende wurde mir dann klar, dass die Responses nicht passend sind. Aus meinem Privaten React Projekt habe ich gelernt, dass Server Responses immer das gleiche Format haben sollten, egal ob success oder fail, weil es die Frontend Programmierung deutlich vereinfacht und man weniger prüfen muss. Leider war das bei mir nicht der Fall und ich musste dann überall neue Exceptions einbauen und auf das neue JSON Response Format umstellen. Es wäre sinnvoller gewesen sich früher Gedanken darüber zu machen... aber aus Fehlern lernt man für gewöhnlich am besten.

5. ZEITABSCHÄTZUNG

Die benötigten Zeiten kann ich leider nur sehr schwer abschätzen. Dadurch, dass ich das Projekt 2 mal neu angefangen habe, weil ich sehr unzufrieden damit war, kann ich nicht wirklich sagen welcher Teil wie lange gedauert hat

Projektteil	Geschätzter Zeitaufwand
Projektplanung im Voraus	ca. 30min (leider viel zu wenig)
Handler Layer und Server implementierung	ca. 15 Stunden
Service Layer	ca. 8 Stunden
Datenbankzugriffe (Repositories)	ca. 15 Stunden
PostgreSQL und Docker	ca. 8 Stunden
Fehlerbehandlung	ca. 20 Stunden
Unit-Tests	ca. 4 Stunden
Debugging & Refactoring	Viel zu viel...
Gesamt	? 100h (kann ich wirklich nicht sagen)

Ich müsste lügen, wenn ich sage, dass ich mir bei diese Zeitabschätzung sicher bin.

6. PERSÖNLICHES FAZIT

Zusammenfassend war das MRP-Projekt für mich eine äußerst lehrreiche und herausfordernde Erfahrung. Besonders die Entscheidung für eine schichtbasierte Architektur hat mir gezeigt, wie wichtig eine saubere Trennung von Verantwortlichkeiten für Wartbarkeit, Erweiterbarkeit und Verständlichkeit eines Backends ist. Durch die klare Aufteilung in Handler-, Service- und Repository-Schicht konnte ich nicht nur die Struktur meines Codes verbessern, sondern auch ein besseres Verständnis für professionelle Backend-Architekturen aneignen.

LINK ZU GITHUB REPO

<https://github.com/deadstx/JavaMRP>